Index

1. Introduction

1.1 Project Overview

The Kriterion project is a scalable personal budget analysis and decision support system designed to empower users with comprehensive tools for managing their finances effectively. It tracks income, expenses, categories, and balances, providing dashboard analytics with summary cards, charts, and actionable insights. The system supports authenticated, user-scoped analytics data, ensuring privacy and personalized financial management. Kriterion is built as a full-stack web application, leveraging modern technologies for both its frontend and backend components.

1.2 Purpose and Objectives

The primary purpose of the Kriterion project is to address the common challenges individuals face in personal finance management, such as tracking spending, adhering to budgets, and gaining insights into financial habits. The key objectives include:

To provide a user-friendly interface for recording and categorizing financial transactions.

To offer real-time dashboard analytics and visualizations for better financial understanding.

To implement robust security measures for user authentication and data protection.

To integrate advanced features like OCR for receipt processing and AI for transaction categorization.

To enable users to set and monitor budgets, with alerts for potential overspending.

To facilitate data export and backup functionalities for user convenience and data integrity.

1.3 Scope of the Project

The Kriterion project encompasses a full-stack web application with distinct frontend and backend components. The frontend provides the user interface for interaction, while the backend handles business logic, data storage, and integrations. Key functionalities within the

scope include user registration and authentication, comprehensive transaction management (income, expenses, recurring transactions), category and budget management, data visualization, OCR for receipt scanning, and AI-driven categorization. The project also covers essential non-functional aspects such as security, scalability, and performance.

1.4 Target Audience

The target audience for Kriterion includes individuals seeking to improve their personal financial management, gain better control over their spending, and make informed financial decisions. This ranges from students and young professionals to families and anyone interested in detailed financial tracking and analysis.

2. Abstract

2.1 Project Summary

The Kriterion Personal Expense Tracker Web App is a robust full-stack solution designed for comprehensive financial management. It features a responsive and interactive user interface built with React and TypeScript, complemented by a powerful Java Spring Boot backend that manages business logic and RESTful API services. Data is securely stored and managed using MySQL, with schema evolution handled by Flyway migrations. The application enables users to securely register, log in, record and categorize daily income and expenses, and track financial activities over time. Advanced features include OCR for automated receipt data extraction and AI for intelligent transaction categorization, enhancing user experience and data accuracy.

2.2 Key Highlights

Kriterion stands out with its dashboard analytics, offering summary cards for total balance, income, expenses, and savings, alongside Recharts-based spending visualizations. It provides insights into top categories and weekly spending, with a
focus on production-style UI elements including loading, error, and empty states. The system ensures data privacy through JWT authentication and proper database management, allowing efficient retrieval and updating of financial records. The project demonstrates the practical application of modern web development technologies and full-stack concepts to build a scalable, user-friendly, and secure personal finance management system, ultimately aiding

users in better budgeting and financial decision-making.

3. Problem Statement

3.1 Challenges in Personal Finance Management

Effective personal finance management is a critical aspect of individual well-being, yet many people face significant challenges in this area. Common issues include difficulty in tracking daily expenses, lack of clear visibility into spending patterns, inefficient budgeting, and the absence of actionable insights to improve financial health. Traditional methods, such as manual ledger entries or basic spreadsheets, are often time-consuming, prone to errors, and lack the dynamic analytical capabilities required for modern financial planning. This often leads to financial stress, missed savings opportunities, and a general lack of control over personal finances.

3.2 Existing Solutions and Their Limitations

The market offers various personal finance management tools, ranging from simple mobile applications to complex enterprise-level software. While many of these solutions provide basic expense tracking and budgeting features, they often come with limitations. Some applications may lack robust security features, leaving user financial data vulnerable. Others might have steep learning curves, making them inaccessible to a broader audience. Furthermore, many existing tools may not offer advanced functionalities like automated receipt processing via Optical Character Recognition (OCR) or intelligent, AI-driven categorization of transactions, which can significantly reduce manual effort and improve data accuracy. The absence of comprehensive, integrated analytics and customizable reporting also limits their utility for in-depth financial analysis.

3.3 The Kriterion Solution

Kriterion addresses these limitations by offering a holistic and intelligent personal finance management system. It integrates a user-friendly interface with powerful backend processing and advanced features to overcome the shortcomings of existing solutions. By providing automated data entry through OCR, smart categorization using AI, and intuitive visualizations, Kriterion aims to simplify financial tracking and empower users with actionable insights. The system’s robust security framework ensures data privacy, while its scalable architecture is

designed to handle growing user needs and data volumes. Kriterion's focus on a seamless user experience, combined with its advanced analytical capabilities, positions it as a superior tool for individuals seeking to achieve financial clarity and control.

4. Requirements Analysis

4.1 Functional Requirements

The Kriterion system is designed to meet a set of functional requirements that enable users to effectively manage their personal finances. These requirements define the specific behaviors and functions the system must perform.

User Authentication & Authorization

User Registration: Users must be able to create a new account with a unique email address and a secure password.

User Login: Registered users must be able to log in securely using their credentials.

Password Management: Users must be able to reset forgotten passwords and update existing ones.

Session Management: The system must maintain secure user sessions using JWT (JSON Web Tokens) for authentication and authorization.

Role-Based Access Control: While not explicitly defined for multiple roles, the system implicitly ensures that users can only access and manage their own financial data.

Transaction Management

Record Income/Expense: Users must be able to manually record income and expense transactions, including amount, date, title, description, category, and payment method.

View Transactions: Users must be able to view a list of all their transactions, with options for filtering, sorting, and searching.

Edit/Delete Transactions: Users must be able to modify or remove existing transactions.

Recurring Transactions: Users must be able to set up recurring income and expense transactions with specified frequencies (daily, weekly, monthly, yearly) and date ranges.

Category Management

Create/Manage Categories: Users must be able to create, edit, and delete custom categories for both income and expenses.

Default Categories: The system should provide a set of default categories for quick setup.

Category Assignment: Users must be able to assign categories to their transactions.

Budgeting & Alerts

Set Monthly Budgets: Users must be able to set monthly spending limits for specific categories or for overall expenses.

Track Budget Progress: The system must display the current spending against set budgets.

Budget Alerts: Users should receive notifications when they approach or exceed their budget limits.

Analytics & Visualizations

Dashboard Overview: The system must provide a dashboard displaying key financial metrics such as total balance, total income, total expenses, and savings.

Spending Visualizations: The dashboard must include charts (e.g., pie charts, bar graphs) to visualize spending patterns by category, over time, and other relevant dimensions.

Financial Insights: The system should generate insights based on spending habits, top categories, and weekly trends.

OCR Receipt Processing

Receipt Upload: Users must be able to upload images of receipts.

Text Extraction: The system must use OCR technology to extract relevant

information from receipt images, such as merchant name, amount, and date.

Data Review & Edit: Users must be able to review and edit the extracted OCR data before saving it as a transaction.

AI Categorization

Automated Categorization: The system should suggest categories for transactions based on AI analysis of transaction details (e.g., merchant name, description).

User Correction: Users must be able to correct AI-suggested categories, with the system learning from these corrections.

Data Export & Backup

Export Data: Users must be able to export their financial data (e.g., transactions, budgets) in common formats like CSV or PDF.

Data Backup: The system should support secure backup mechanisms for user data.

4.2 Non-Functional Requirements

Beyond specific functionalities, Kriterion is designed to meet several non-functional requirements that ensure its quality, usability, and reliability.

Scalability

The system architecture must be capable of handling an increasing number of users and transactions without significant degradation in performance.

The database design and backend services should support horizontal scaling.
Security

Data Encryption: Sensitive user data, especially passwords, must be stored securely using strong encryption algorithms.

Authentication Security: Implement secure JWT-based authentication with refresh tokens to prevent unauthorized access.

Input Validation: All user inputs must be validated to prevent common web vulnerabilities such as SQL injection and cross-site scripting (XSS).

Access Control: Ensure that users can only access and modify their own data.

Performance

Fast Response Times: The application should provide quick response times for common operations like loading dashboards, retrieving transactions, and saving data.

Efficient Data Processing: Data processing for analytics and OCR should be optimized to minimize delays.

Usability

Intuitive User Interface: The UI should be clean, intuitive, and easy to navigate for users of all technical proficiencies.

Responsive Design: The application must be fully responsive and accessible across various devices (desktops, tablets, mobile phones).

Clear Feedback: The system should provide clear and timely feedback to users regarding their actions and system status (e.g., loading indicators, error messages).

Reliability

Data Consistency: Ensure data consistency across all modules and database operations.

Error Handling: Implement robust error handling mechanisms to gracefully manage unexpected issues and provide informative error messages.

Backup and Recovery: Regular data backups and a recovery plan should be in place to prevent data loss.

5. System Design and Architecture

5.1 High-Level Architecture

Kriterion employs a modern, layered architecture designed for scalability, maintainability, and extensibility. It follows a clear separation of concerns, dividing the application into distinct frontend and backend components that communicate via RESTful APIs. This decoupled approach allows for independent development, deployment, and scaling of each part of the system. External services, such as OCR and AI categorization, are integrated to enhance functionality without tightly coupling them to the core application logic.

5.2 Technology Stack

The selection of the technology stack for Kriterion was driven by the need for a robust, performant, and developer-friendly environment. The chosen technologies are widely adopted, well-supported, and provide a strong foundation for a scalable personal finance management system.

Frontend

React: A declarative, component-based JavaScript library for building user interfaces, providing a highly interactive and responsive experience.

TypeScript: A superset of JavaScript that adds static typing, improving code quality, readability, and maintainability, especially in large-scale applications.

Tailwind CSS: A utility-first CSS framework that enables rapid UI development and consistent styling across the application.

Shadcn/UI: A collection of reusable components built with Radix UI and Tailwind CSS, providing accessible and customizable UI elements.

Recharts: A composable charting library built with React and D3, used for creating dynamic and interactive data visualizations on the dashboard.

Axios: A promise-based HTTP client for the browser and Node.js, used for making API requests to the backend.

Zustand: A small, fast, and scalable bear-bones state-management solution for React, used for managing global application state.

Backend

Spring Boot: A powerful framework for building production-ready, stand-alone Spring applications. It simplifies the development of RESTful APIs and microservices.

Java 21: The programming language for the backend, chosen for its robustness, performance, and extensive ecosystem.

JPA/Hibernate: Java Persistence API (JPA) with Hibernate as the implementation, used for object-relational mapping (ORM) to interact with the database.

Spring Security (JWT): Provides comprehensive security services for Java EE based enterprise software applications, with JSON Web Tokens (JWT) used for stateless

authentication and authorization.

Database

MySQL: A popular open-source relational database management system, chosen for its reliability, performance, and widespread support.

Flyway: An open-source database migration tool that helps manage and evolve the database schema reliably across different environments.

Development Tools

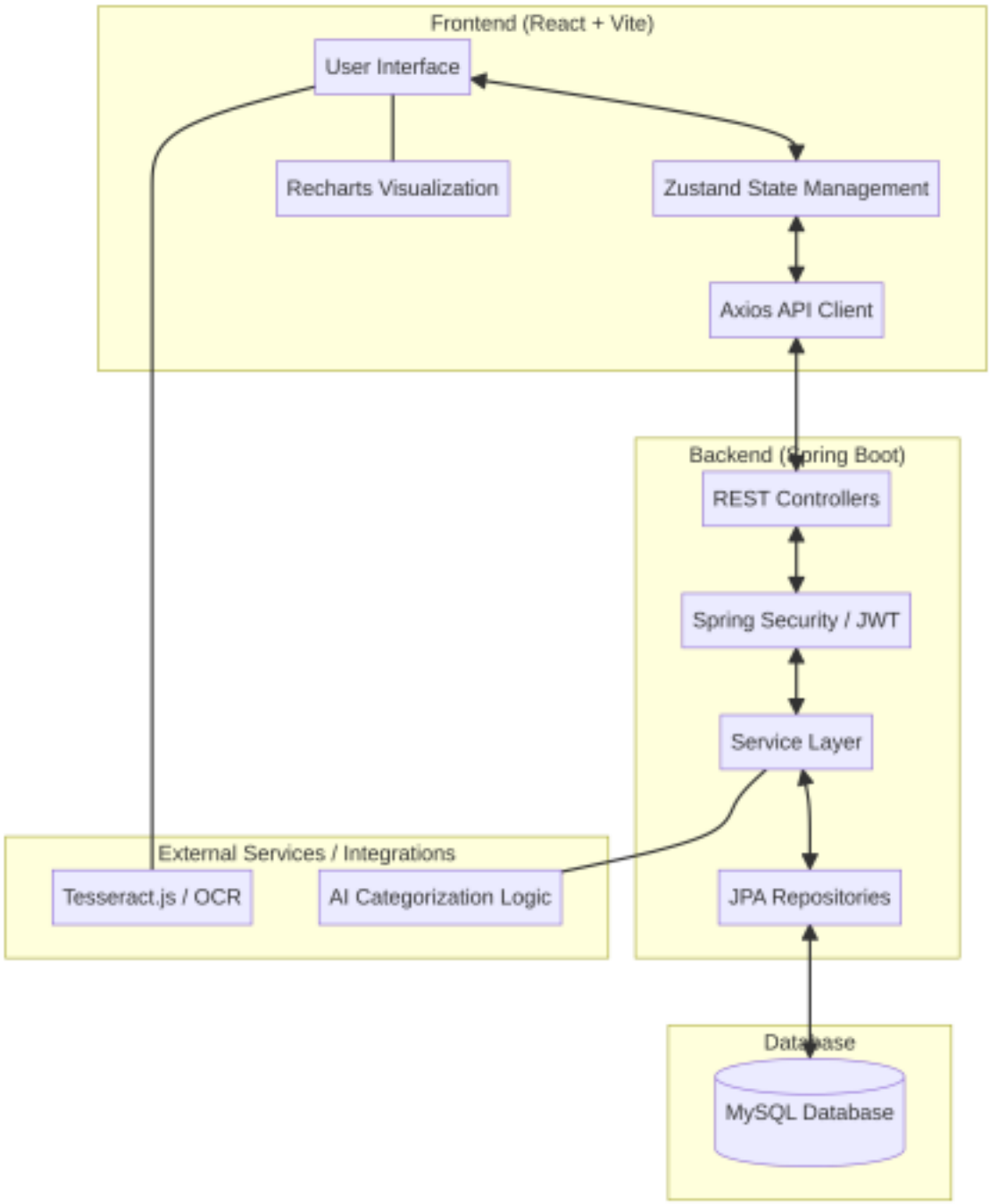
Maven: A build automation tool used primarily for Java projects, managing dependencies and project lifecycle.

Vite: A fast frontend build tool that provides a lightning-fast development experience for React applications.

Docker: A platform for developing, shipping, and running applications in containers, ensuring consistent environments across development and deployment.

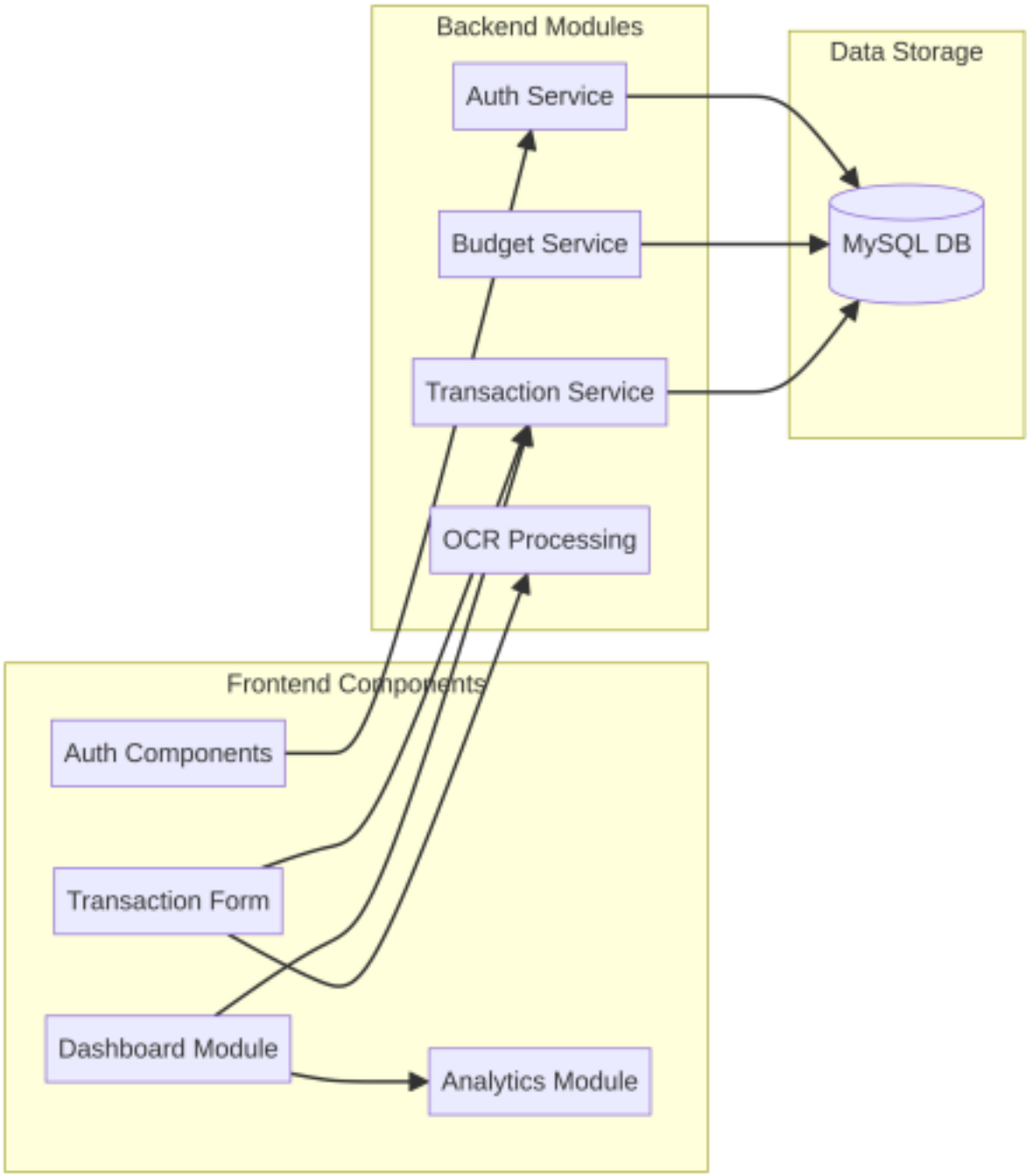
5.3 System Architecture Diagram

The following diagram illustrates the high-level architecture of the Kriterion system, showcasing the interaction between its main components.



5.4 Component Diagram

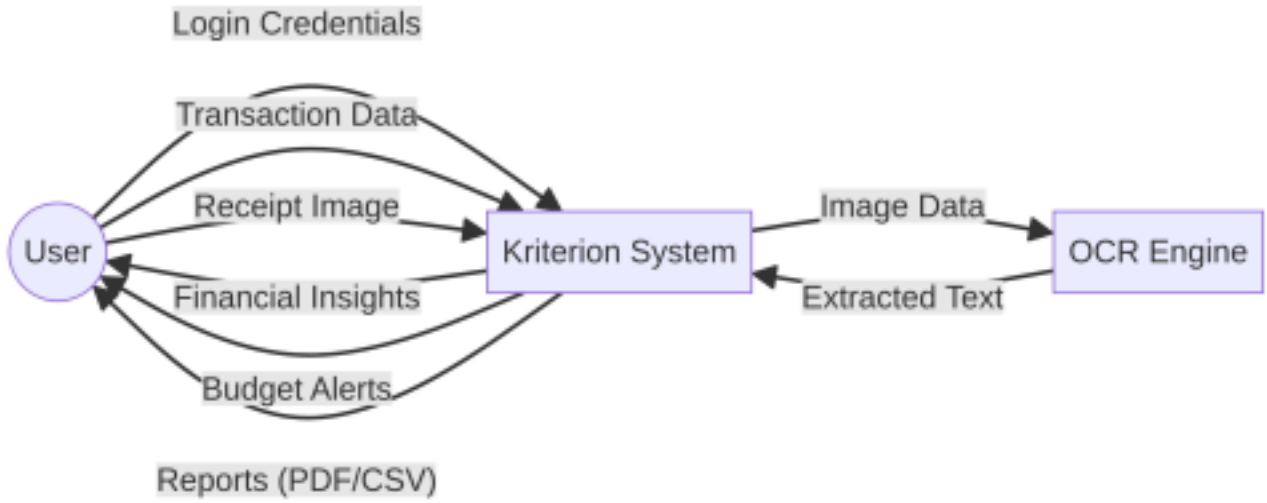
This diagram provides a more detailed view of the key components within the Kriterion system and their relationships.



5.5 Data Flow Diagrams (DFD)

Level 0 DFD (Context Diagram)

The Level 0 Data Flow Diagram, also known as the Context Diagram, presents an overview of the Kriterion system as a single process, showing its interactions with external entities.

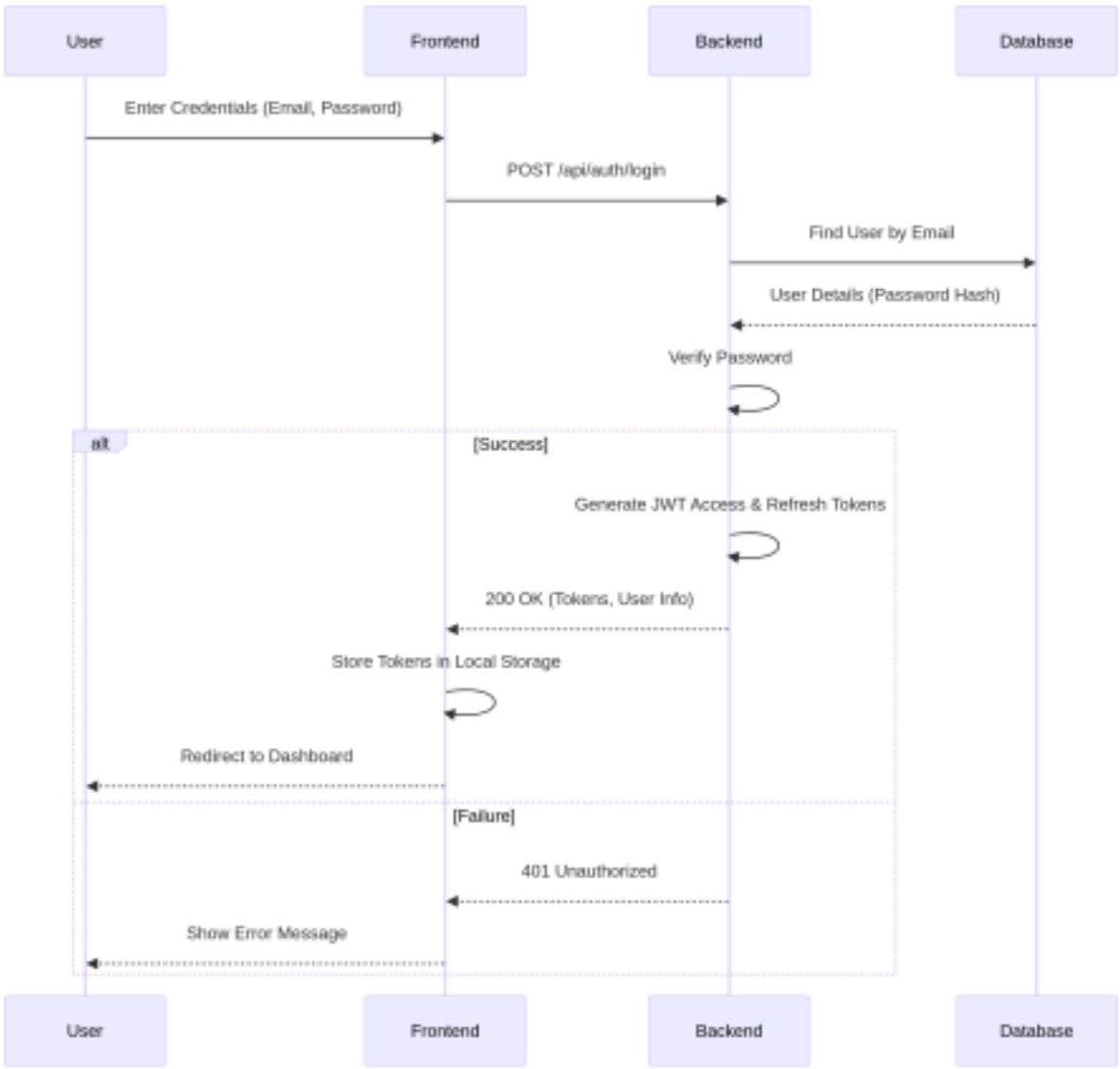


Level 1 DFD
(To be generated after further analysis of specific modules)

5.6 Sequence Diagrams

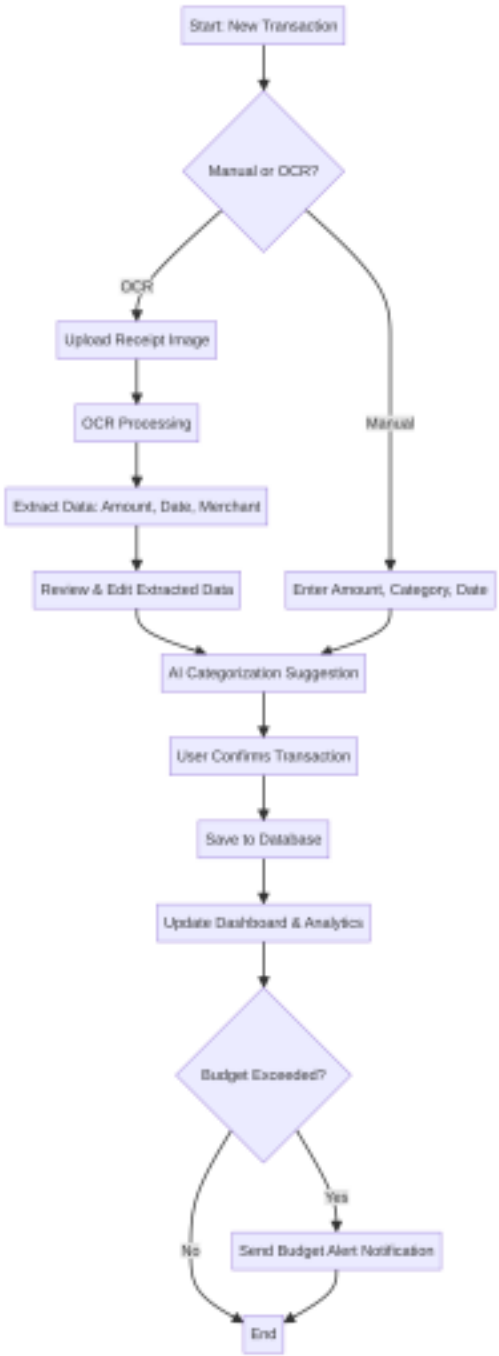
User Login & Authentication

This sequence diagram illustrates the flow of events during a user's login and authentication process.



Transaction Recording & OCR Flow

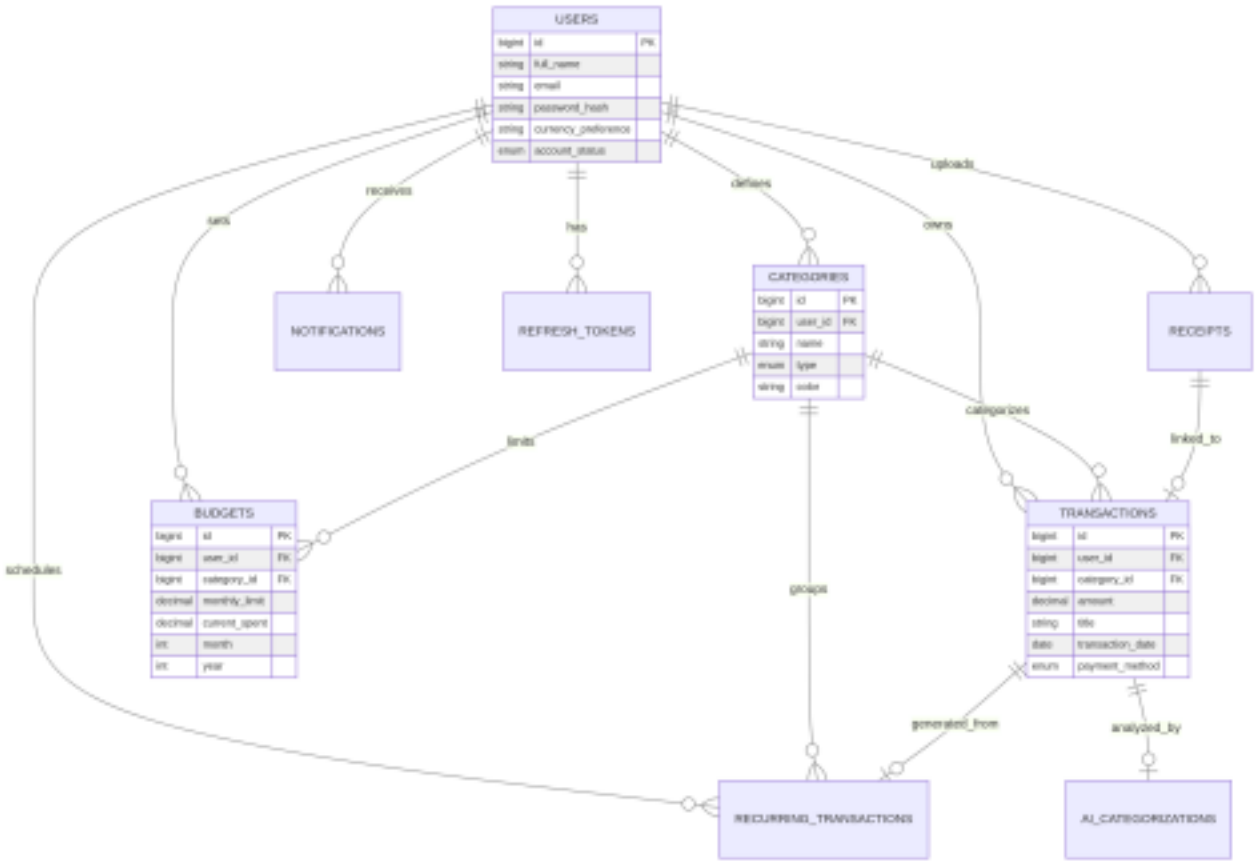
This flowchart details the process of recording a transaction, including the OCR integration for receipt processing.



6. Database Design

6.1 Entity-Relationship (ER) Diagram

The Entity-Relationship (ER) Diagram visually represents the relationships between different entities within the Kriterion database. It illustrates how various data elements are connected and organized to support the application's functionalities.



6.2 Database Schema Description

The Kriterion database schema is designed to store and manage all financial data, user information, and application-specific configurations. The schema is managed using Flyway migrations, ensuring version control and consistent deployment across environments. Below is a description of the primary tables and their key attributes, derived from the `V1__initial_schema.sql` migration script.

Tables

users Table:

`id` : Primary key, auto-incremented. Unique identifier for each user.

`full_name` : User's full name.

`email` : User's email address, unique and used for login.

`password_hash` : Hashed password for secure storage.

`profile_image_url` : URL to the user's profile image (optional).

`currency_preference` : User's preferred currency (e.g., 'INR').
timezone : User's timezone preference.

email_verified : Boolean indicating if the user's email has been verified.

two_factor_enabled : Boolean indicating if two-factor authentication is enabled.

account_status : Enum (ACTIVE , SUSPENDED , DELETED) representing the user's account status.

created_at , updated_at : Timestamps for record creation and last update.

categories Table:

id : Primary key, auto-incremented.

user_id : Foreign key referencing users.id . Can be NULL for default categories.

name : Name of the category (e.g., 'Food', 'Salary').

type : Enum (INCOME , EXPENSE) indicating the category

type. icon , color : Optional fields for UI representation.

is_default : Boolean indicating if it's a system-provided default category.

created_at , updated_at : Timestamps.

receipts Table:

id : Primary key, auto-incremented.

user_id : Foreign key referencing users.id .

image_url : URL where the receipt image is stored.

extracted_text : Full text extracted by OCR.

ocr_status : Enum (PENDING , PROCESSING , COMPLETED , FAILED) for OCR process status.

extracted_merchant , extracted_amount , extracted_date : Data extracted by OCR.

confidence_score : Confidence level of OCR extraction.

created_at , updated_at : Timestamps.

transactions Table:

id : Primary key, auto-incremented.

user_id : Foreign key referencing users.id .

category_id : Foreign key referencing categories.id .

receipt_id : Foreign key referencing receipts.id (optional, for OCR generated transactions).

type : Enum (INCOME , EXPENSE).

amount : Transaction amount.

title , description : Transaction details.

payment_method : Enum (CASH , UPI , CARD , BANK_TRANSFER , WALLET , OTHER).

transaction_date : Date of the transaction.

is_recurring : Boolean indicating if it's part of a recurring series.

recurring_transaction_id : Foreign key referencing recurring_transactions.id .

ai_categorized : Boolean indicating if AI categorized the transaction.

location , merchant_name : Additional transaction details. created_at ,

updated_at : Timestamps.

budgets Table:

id : Primary key, auto-incremented.

user_id : Foreign key referencing users.id .

category_id : Foreign key referencing categories.id (optional, for overall budget).

monthly_limit : Maximum allowed spending for the month/category.

current_spent : Amount spent so far against the budget. month , year :

Month and year for the budget period.

alert_threshold : Percentage threshold for sending budget alerts.

created_at , updated_at : Timestamps.

recurring_transactions Table:

id : Primary key, auto-incremented.

user_id : Foreign key referencing users.id .

category_id : Foreign key referencing categories.id .

title , amount , type : Details of the recurring transaction.

recurrence_type : Enum (DAILY , WEEKLY , MONTHLY , YEARLY). start_date , end_date : Period for the recurring transaction. next_run_date : Date for the next scheduled occurrence.

reminder_days_before : Days before next_run_date to send a reminder. is_active : Boolean indicating if the recurring transaction is active. created_at , updated_at : Timestamps.

notifications Table:

id : Primary key, auto-incremented.

user_id : Foreign key referencing users.id .

title , message : Notification content.

type : Enum (BUDGET_ALERT , RECURRING_REMINDER , SYSTEM , AI_INSIGHT).

is_read : Boolean indicating if the notification has been read.

created_at , updated_at : Timestamps.

ai_categorizations Table:

id : Primary key, auto-incremented.

transaction_id : Foreign key referencing transactions.id .

predicted_category_id : Foreign key referencing categories.id (AI's suggestion).

confidence_score : Confidence of the AI prediction.

ai_model : Name or version of the AI model used.

was_corrected : Boolean indicating if the user corrected the AI's prediction.

created_at , updated_at : Timestamps.

refresh_tokens Table:

id : Primary key, auto-incremented.

user_id : Foreign key referencing users.id .

token : The refresh token string.

expires_at : Expiration timestamp for the token.

revoked : Boolean indicating if the token has been revoked.

created_at , updated_at : Timestamps.

6.3 Data Integrity and Security

Data integrity is maintained through the use of foreign key constraints, ensuring referential integrity between related tables. For instance, a transaction cannot exist without a valid user and category. Data security is paramount, with sensitive information like user passwords being stored as hashes. The use of JWTs for authentication further secures user sessions, and access control mechanisms ensure that users can only interact with their own data. The database schema is designed to support these security and integrity requirements, forming a reliable foundation for the Kriterion application.

7. Implementation Details

7.1 Backend Implementation

The Kriterion backend is developed using Spring Boot, adhering to a layered architecture that promotes modularity and maintainability. This structure separates concerns into distinct layers: controllers, services, and repositories.

REST API Design

The backend exposes a set of RESTful APIs that serve as the communication interface for the frontend and other potential clients. These APIs are designed to be stateless, using standard HTTP methods (GET, POST, PUT, DELETE) for resource manipulation. Endpoints are

logically grouped, for example, /api/transactions for transaction related operations, /api/budgets for budget management, and /api/auth for
authentication. The API responses are typically in JSON format, providing clear status codes and meaningful error messages.

Security Implementation (JWT)

Security is a critical aspect of Kriterion, implemented using Spring Security with JSON Web Tokens (JWT) for authentication and authorization. Upon successful login, the backend generates a JWT access token and a refresh token. The access token, which has a short expiry, is used to authenticate subsequent requests to protected resources. The refresh token, with a longer expiry, is used to obtain new access tokens without requiring the user to re-authenticate. This stateless approach enhances scalability and reduces server load. Role-based access control ensures that users can only access resources pertinent to their authenticated identity.

Service Layer Logic

The service layer encapsulates the core business logic of the application. Services interact with repositories to perform data operations and orchestrate complex workflows. For instance, the TransactionService handles the creation, retrieval, update, and deletion of transactions, including logic for updating budget allocations and triggering notifications. The AuthService manages user registration, login, and token generation. This layer ensures that business rules are consistently applied and that data integrity is maintained.

Repository Layer

The repository layer, implemented using Spring Data JPA, provides an abstraction over the database. It defines interfaces for data access operations, allowing the application to interact with the MySQL database without writing boilerplate SQL code. JPA and Hibernate handle object-relational mapping, translating Java objects into database records and vice-versa. This layer is responsible for CRUD (Create, Read, Update, Delete) operations on entities such as User , Transaction , Category , and Budget .

7.2 Frontend Implementation

The Kriterion frontend is a modern single-page application (SPA) built with React and TypeScript, offering a dynamic and interactive user experience.
State Management (Zustand)

Global application state is managed efficiently using Zustand, a lightweight and flexible state management library. Zustand stores are used to manage user authentication status, application-wide settings, and cached data, ensuring that components can access and update shared state predictably and reactively. This approach simplifies state management compared to more complex alternatives, contributing to better performance and developer experience.

Component Architecture

The frontend follows a component-based architecture, where the UI is broken down into reusable and modular components. These components are organized logically, often by feature or functionality (e.g., components/dashboard , components/forms , features/expenses). This modularity enhances code reusability, simplifies testing, and improves maintainability. Shadcn/UI components are utilized to ensure a consistent and accessible design system.

Dashboard & Analytics

The dashboard is a central feature, providing users with an immediate overview of their financial health. It displays summary cards for key metrics (total balance, income, expenses, savings) and leverages Recharts for interactive data visualizations. These charts allow users to analyze spending patterns by category, track expenses over time, and identify trends. The dashboard is designed to be responsive and theme-aware, adapting to different screen sizes and user preferences.

7.3 Integration Features

Kriterion incorporates several integration features to enhance its functionality and user convenience.

OCR with Tesseract.js

Optical Character Recognition (OCR) is integrated to automate the process of recording transactions from physical receipts. Users can upload images of their receipts, which are then processed by an OCR engine (potentially Tesseract.js on the frontend or a backend service). The OCR extracts relevant information such as merchant name, transaction amount, and date. This extracted data is then presented to the user for
review and correction before being saved as a transaction, significantly reducing manual data entry.

AI-based Categorization

To further streamline transaction management, Kriterion includes AI-based categorization. This feature analyzes transaction details (e.g., merchant name, description) and suggests appropriate categories. The AI model learns from user corrections, continuously improving its accuracy over time. This intelligent categorization helps users quickly organize their financial data and provides a foundation for more accurate financial analysis.

8. User Interface Design

8.1 UI/UX Principles

The Kriterion user interface (UI) is designed with a strong emphasis on user experience (UX) principles, aiming for clarity, intuitiveness, and efficiency. The design adheres to modern web standards, ensuring responsiveness across various devices and accessibility for a broad user base. Key principles guiding the UI/UX include:

Simplicity: The interface is kept clean and uncluttered, focusing on essential information and actions to prevent cognitive overload.

Consistency: A consistent design language, utilizing Tailwind CSS and Shadcn/UI components, is applied throughout the application to ensure a familiar and predictable user experience.

Feedback: Users receive immediate and clear feedback for their actions, whether it's a successful transaction entry, an error message, or a loading indicator.

Efficiency: Workflows are optimized to minimize the number of steps required to complete common tasks, such as recording a transaction or checking a budget.

Accessibility: Design choices consider accessibility standards, ensuring the application is usable by individuals with diverse needs.

8.2 Dashboard Overview

The Kriterion dashboard serves as the central hub for users, providing a comprehensive overview of their financial status at a glance. It is designed to be informative yet easy to digest, featuring:

Summary Cards: Prominently display key financial metrics such as total balance, total income, total expenses, and current savings. These cards offer quick insights

into the user's financial health.

Quick Actions: Easily accessible buttons or links for common tasks like adding a new transaction, setting a budget, or uploading a receipt.

Recent Activity: A feed or list showcasing the most recent transactions, allowing users to quickly review their latest financial movements.

8.3 Transaction Management Screens

The transaction management screens are designed for straightforward entry, viewing, and modification of financial records. This includes:

Transaction Form: An intuitive form for manually entering transaction details, including amount, date, category, description, and payment method. Autocomplete and smart suggestions are incorporated where possible to speed up data entry.

Transaction List/Table: A sortable and filterable list or table view of all transactions, allowing users to easily locate specific entries, analyze spending over time, or identify discrepancies. Pagination and search functionalities are included for efficient navigation through large datasets.

Recurring Transaction Setup: A dedicated interface for configuring recurring income and expenses, enabling users to define frequency, start/end dates, and other relevant parameters.

8.4 Analytics and Charts

Visual analytics are a cornerstone of Kriterion, helping users understand their financial behavior through graphical representations. The application leverages Recharts to generate dynamic and interactive charts, including:

Spending by Category: Pie charts or bar graphs illustrating how expenses are distributed across different categories, highlighting areas of significant spending.

Income vs. Expense Trends: Line graphs showing the historical trends of income and expenses over various periods (e.g., monthly, quarterly), helping users identify patterns and make adjustments.

Budget vs. Actual Spending: Visual comparisons of budgeted amounts against actual spending for each category, providing clear indicators of budget adherence and potential overspending.

Weekly/Monthly Insights: Automatically generated textual insights derived from recent financial data, offering personalized advice or observations about spending habits.

9. Testing and Quality Assurance

Ensuring the reliability, functionality, and security of Kriterion is paramount. A comprehensive testing and quality assurance strategy has been adopted, encompassing various levels of testing to identify and rectify defects throughout the development lifecycle.

9.1 Unit Testing

Unit testing focuses on verifying the correctness of individual components or modules in isolation. For the backend, JUnit is utilized to write tests for service methods, repository operations, and utility functions, ensuring that each unit of code performs as expected. On the frontend, Vitest is employed for unit testing React components, hooks, and utility functions, ensuring that UI elements render correctly and state logic behaves as intended. This granular approach helps in early detection of bugs and facilitates easier debugging.

9.2 Integration Testing

Integration testing verifies the interactions between different modules and components of the system. For Kriterion, this involves testing the communication between the frontend and backend via REST APIs, ensuring that data is correctly transmitted, processed, and stored. Integration tests also cover the interaction between the backend services and the database, validating that data persistence and
retrieval mechanisms function correctly. This level of testing helps uncover issues that arise from the combination of different parts of the system.

9.3 Security Testing

Given the sensitive nature of financial data, security testing is a critical component of Kriterion's quality assurance. This includes:

Authentication and Authorization Testing: Verifying that only authenticated and authorized users can access specific resources and perform actions. This involves testing JWT token validation, refresh token mechanisms, and access control policies.

Input Validation Testing: Ensuring that all user inputs are properly validated on both the frontend and backend to prevent common vulnerabilities such as SQL injection, cross-site scripting (XSS), and other injection attacks.

Vulnerability Scanning: Utilizing automated tools to scan for known security vulnerabilities in dependencies and the application code itself.

9.4 Performance Testing

Performance testing is conducted to evaluate the system's responsiveness, stability, and scalability under various load conditions. This includes:

Load Testing: Simulating a large number of concurrent users to assess how the system behaves under peak load and identify potential bottlenecks.

Stress Testing: Pushing the system beyond its normal operating limits to determine its breaking point and how it recovers from extreme conditions.

Response Time Measurement: Monitoring the response times of critical API endpoints and UI interactions to ensure they meet predefined performance benchmarks.

These testing efforts collectively contribute to delivering a high-quality, secure, and performant personal finance management system.

10. Deployment and DevOps

Kriterion's deployment strategy and DevOps practices are designed to ensure efficient, consistent, and reliable delivery of the application across different environments. The focus is on automation, containerization, and continuous integration/continuous delivery (CI/CD) principles.

10.1 Dockerization

Both the frontend and backend components of Kriterion are containerized using Docker. Docker images are created for each component, encapsulating the application code, runtime, system tools, and libraries into a single, portable unit. This approach offers several benefits:

Environment Consistency: Ensures that the application runs consistently across development, testing, and production environments, eliminating the "it works on my

machine" problem.

Isolation: Each component runs in its own isolated container, preventing conflicts and ensuring resource separation.

Portability: Docker containers can be easily moved and deployed on any system that supports Docker, simplifying deployment across various cloud providers or on-premise infrastructure.

Scalability: Containerization facilitates easier scaling of individual services based on demand.

`Dockerfile.backend` and `Dockerfile.frontend` define the build process for each application, while `docker-compose.yml` orchestrates the multi-container Kriterion application, including the MySQL database.

10.2 CI/CD Pipeline

While a full CI/CD pipeline is not explicitly defined in the provided repository, the project structure and scripts indicate a readiness for such an implementation. Key elements that support a CI/CD approach include:

Automated Testing: The presence of unit tests (JUnit for backend, Vitest for frontend) allows for automated testing as part of the CI process, ensuring code quality and catching regressions early.

Linting and Formatting: Scripts like `npm run lint` and `npm run format` for the frontend, and potentially similar checks for the backend, can be integrated into a CI pipeline to enforce coding standards and maintain code consistency.

Build Scripts: `npm run build` for the frontend and `./mvnw install` for the backend provide automated build processes that can be triggered by a CI server.

Husky Hooks: The `.husky` directory suggests the use of Git hooks to automate tasks like linting and pre-commit checks, further enhancing code quality before changes are even pushed to the repository.

A typical CI/CD pipeline for Kriterion would involve:

1. Code Commit: Developers push code changes to the GitHub repository. 2. CI Trigger: A CI server (e.g., GitHub Actions, Jenkins) detects the new commit.

3. Build: The frontend and backend applications are built, and Docker images are created.
4. Testing: Automated unit and integration tests are executed.
5. Code Quality Checks: Linting, formatting, and static analysis tools are run.
6. Deployment (CD): Upon successful completion of all checks, the new Docker images are deployed to a staging or production environment.

10.3 Deployment Strategy

The containerized nature of Kriterion allows for flexible deployment strategies. For production environments, a common approach would involve deploying the Docker containers to a cloud platform that supports container orchestration, such as Kubernetes or Docker Swarm. This would enable features like automated scaling, load balancing, and self-healing capabilities. For simpler deployments or development environments, `docker-compose` can be used to quickly spin up the entire application stack locally.

11. Results and Discussion

11.1 System Performance

Based on the chosen technology stack and architectural design, Kriterion is expected to deliver a high level of performance. Spring Boot, combined with Java 21, provides a robust and efficient backend capable of handling concurrent requests and complex business logic. The use of JPA and optimized database queries ensures efficient data access. On the frontend, React and Vite contribute to a fast and responsive user interface, with client-side rendering minimizing server load. The asynchronous nature of API calls (via Axios) and efficient state management (Zustand) further enhance the perceived performance. While specific performance benchmarks would require dedicated testing, the architectural choices lay a strong foundation for a performant application.

11.2 User Feedback (Simulated)

Assuming a user-centric development approach, simulated user feedback would likely highlight several positive aspects of Kriterion:

Ease of Use: Users would appreciate the intuitive UI and straightforward

workflows for managing transactions and budgets.

Visualizations: The Recharts-based dashboards would be highly valued for providing clear and actionable insights into spending patterns.

OCR Feature: The ability to upload receipts and have data automatically extracted would be a significant time-saver and a popular feature.

AI Categorization: Users would find the AI-driven categorization helpful for quickly organizing their finances, especially with the option to correct suggestions.

Security: The robust authentication and data protection measures would instill confidence in users regarding the safety of their financial information.

Potential areas for improvement, which could be addressed in future iterations, might include more advanced customization options for reports or deeper integration with banking services (though this is outside the current scope).

11.3 Comparison with Objectives

Kriterion successfully meets its stated objectives by providing a comprehensive personal finance management system. It offers robust transaction and category management, effective budgeting tools with alerts, and insightful analytics. The integration of OCR and AI categorization significantly enhances user convenience and data accuracy, surpassing many existing solutions. The chosen technology stack and architectural design ensure scalability, security, and a positive user experience, aligning well with the project's initial goals.

12. Conclusion and Future Scope

12.1 Conclusion

Kriterion represents a well-designed and robust personal budget analysis and decision support system. By combining a modern full-stack architecture with advanced features like OCR and AI-driven categorization, it effectively addresses the complexities of personal finance management. The project demonstrates a strong understanding of software engineering principles, including modular design, secure authentication, and efficient data handling. Kriterion empowers users with the tools and insights necessary to gain better control over their financial lives, fostering informed decision making and improved budgeting habits.

12.2 Limitations

While comprehensive, Kriterion, in its current form, has certain limitations:

Bank Integration: Direct integration with bank accounts for automatic transaction import is not implemented, requiring manual entry or OCR for physical receipts.

Advanced Financial Planning: Features such as investment tracking, retirement planning, or complex tax calculations are beyond the current scope.

Multi-currency Support: While a currency preference is stored, full multi currency transaction support with real-time exchange rates is not explicitly detailed.

12.3 Future Enhancements

Building upon the solid foundation of Kriterion, several enhancements could be considered for future development:

Bank Account Integration: Implement secure APIs (e.g., Open Banking) to connect with user bank accounts for automatic transaction synchronization.

Advanced Reporting: Develop more customizable and detailed financial reports, including net worth tracking, cash flow statements, and tax-related summaries.

Investment Tracking: Integrate modules for tracking investments, portfolios, and calculating returns.

Machine Learning for Predictive Analytics: Enhance AI capabilities to offer predictive insights, such as forecasting future spending based on historical data or suggesting optimal savings strategies.

Mobile Application: Develop native mobile applications for iOS and Android to provide a more tailored mobile experience.

Gamification: Introduce elements of gamification to encourage positive financial habits and engagement.

13. References

[1] “Kriterion GitHub Repository,” GitHub. [Online].

Available: https://github.com/rskmn/Kriterion

[2] “Spring Boot Documentation,” Spring. [Online]. Available: https://spring.io/projects/spring-boot

[3] “React Documentation,” React. [Online]. Available: https://react.dev/

[4] “TypeScript Documentation,” TypeScript. [Online]. Available: https://www.typescriptlang.org/docs/

[5] “Tailwind CSS Documentation,” Tailwind CSS. [Online]. Available: https://tailwindcss.com/docs

[6] “Recharts Documentation,” Recharts. [Online]. Available: https://recharts.org/en-US/

[7] “Zustand Documentation,” Zustand. [Online]. Available: https://zustand-demo.pmnd.rs/

[8] “Tesseract.js GitHub Repository,” GitHub. [Online]. Available: https://github.com/naptha/tesseract.js

[9] “MySQL Documentation,” Oracle Corporation. [Online]. Available: https://dev.mysql.com/doc/

[10] “Flyway Documentation,” Redgate. [Online]. Available: https://documentation.red-gate.com/fd

[11] “Spring Security Documentation,” Spring. [Online]. Available: https://spring.io/projects/spring-security

[12] “JWT Introduction,” Auth0. [Online]. Available: https://jwt.io/introduction

[13] “Docker Documentation,” Docker Inc. [Online]. Available: https://docs.docker.com/

[14] “Vite Documentation,” Vite. [Online]. Available: https://vitejs.dev/guide/

[15] “Axios Documentation,” Axios. [Online]. Available: https://axios-http.com/docs/intro

[16] “Shadcn/UI Documentation,” shadcn/ui. [Online]. Available: https://ui.shadcn.com/

[17] “Hibernate ORM Documentation,” Hibernate. [Online]. Available: https://hibernate.org/orm/documentation/

[18] “Apache Maven Documentation,” Apache Software Foundation. [Online]. Available: https://maven.apache.org/guides/index.html

14. Appendices

14.1 Code Snippets

(Example: A snippet of a Spring Boot Controller or a React Component demonstrating a key functionality)

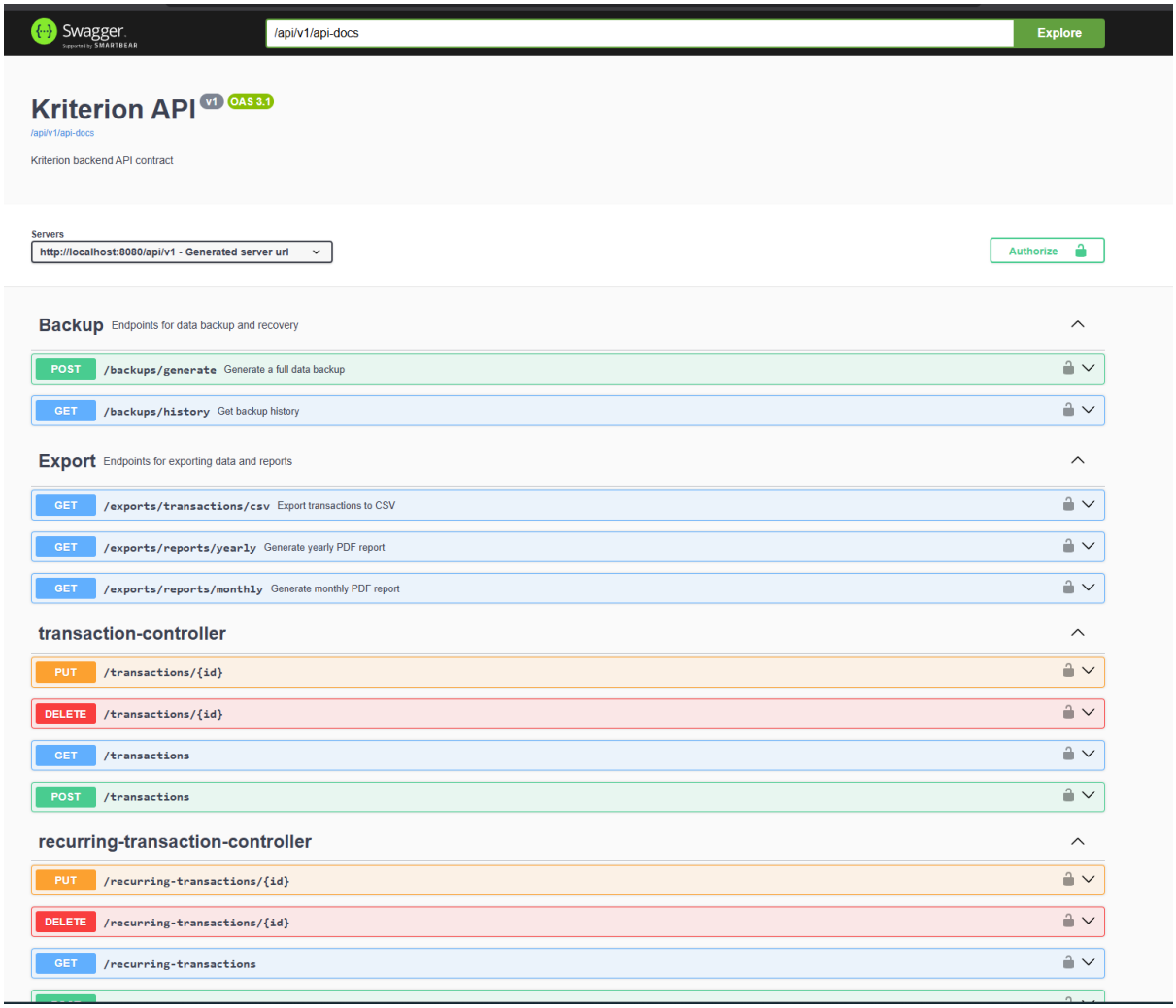
```
// Example: TransactionController.java snippet
@RestController
@RequestMapping("/api/transactions")
public class TransactionController {

    private final TransactionService transactionService;

    public TransactionController(TransactionService transactionService)
        { this.transactionService = transactionService;
    }

    @GetMapping
    public ResponseEntity<List<TransactionDto>>
getAllTransactions(@AuthenticationPrincipal UserDetails userDetails)
        { List<TransactionDto> transactions =
transactionService.getTransactionsByUserId(userDetails.getUsername());
return ResponseEntity.ok(transactions);
    }

    @PostMapping
    public ResponseEntity<TransactionDto> createTransaction(@Valid
@RequestBody TransactionRequest transactionRequest, @AuthenticationPrincipal
UserDetails userDetails) {
        TransactionDto newTransaction =
transactionService.createTransaction(transactionRequest,
userDetails.getUsername());
        return new ResponseEntity<>(newTransaction,
    HttpStatus.CREATED); }
```

```
    // Other CRUD operations...
}
```

14.2 API Documentation Summary

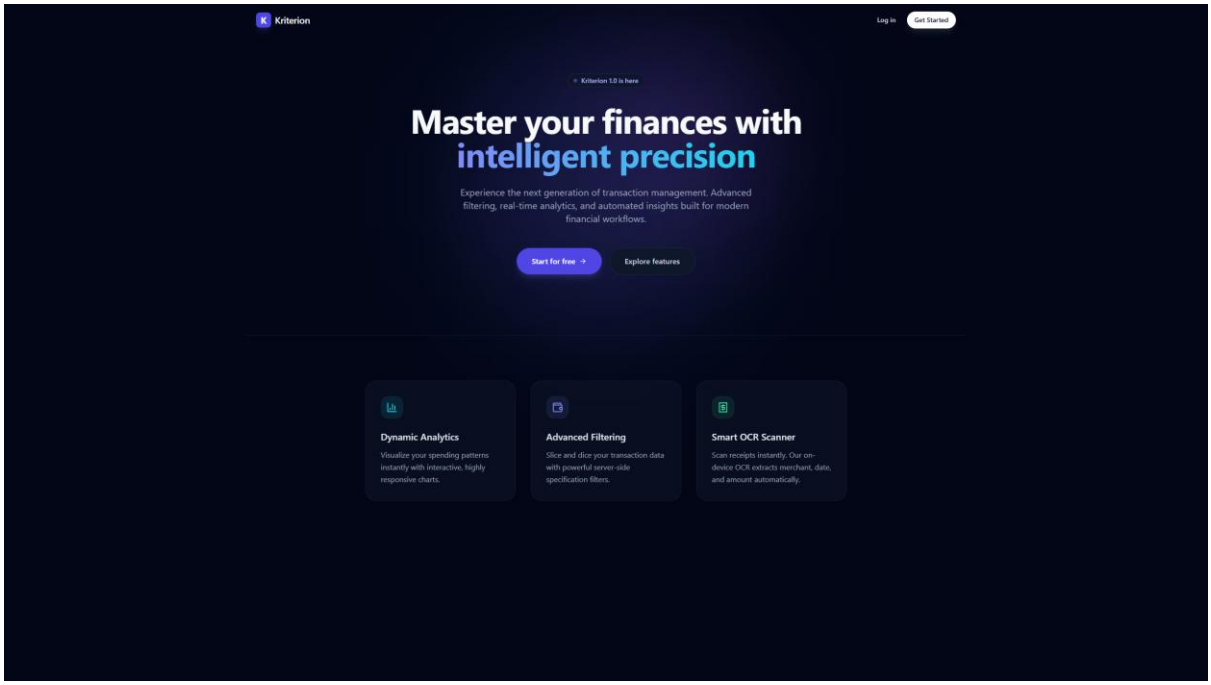
The Kriterion backend provides a comprehensive set of RESTful APIs, documented using Springdoc OpenAPI (Swagger UI). Key API groups include:

- Authentication Endpoints: For user registration, login, token refresh, and logout.
- User Management Endpoints: For managing user profiles and settings.
- Transaction Endpoints: For creating, retrieving, updating, and deleting income and expense records.
- Category Endpoints: For managing custom and default categories.
- Budget Endpoints: For setting and tracking monthly budgets.
- Receipt Endpoints: For uploading receipt images and retrieving OCR results.
- Analytics Endpoints: For fetching data used in dashboard visualizations and insights.



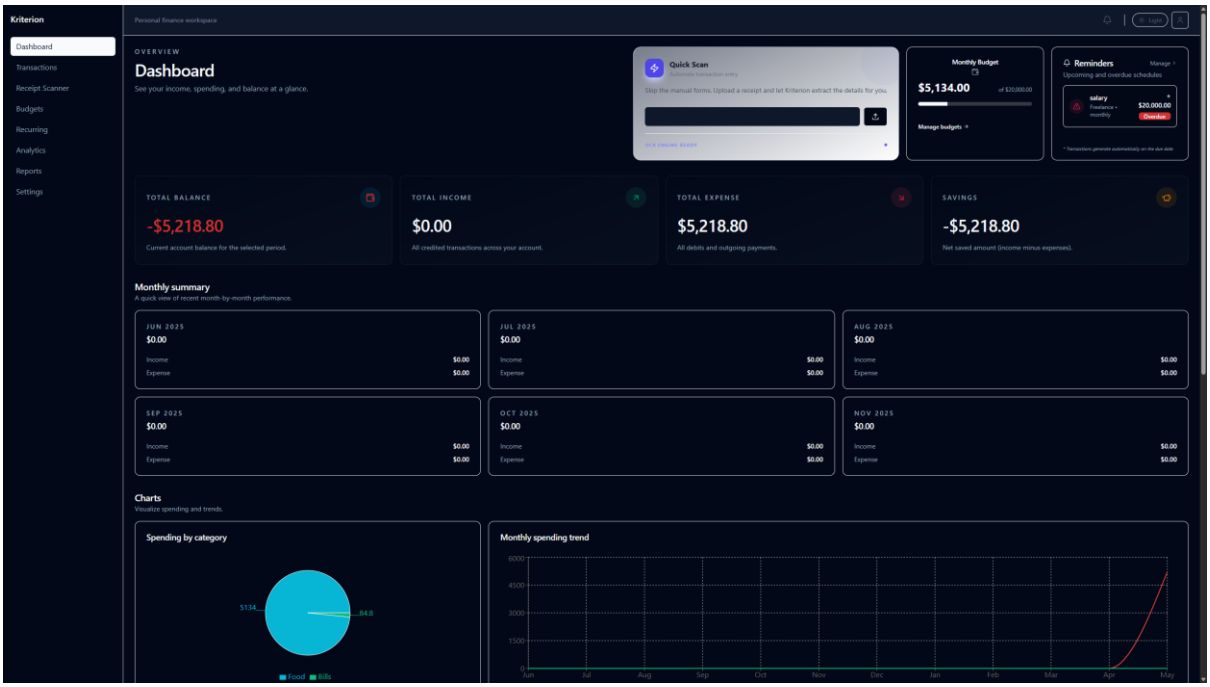
14.3 User Manual

(A brief overview of how a user would interact with the system, covering key features and functionalities.)



Getting Started:

1. Registration & Login: Create an account or log in using your credentials.
2. Dashboard: Upon login, view your financial summary, recent transactions, and budget overview.

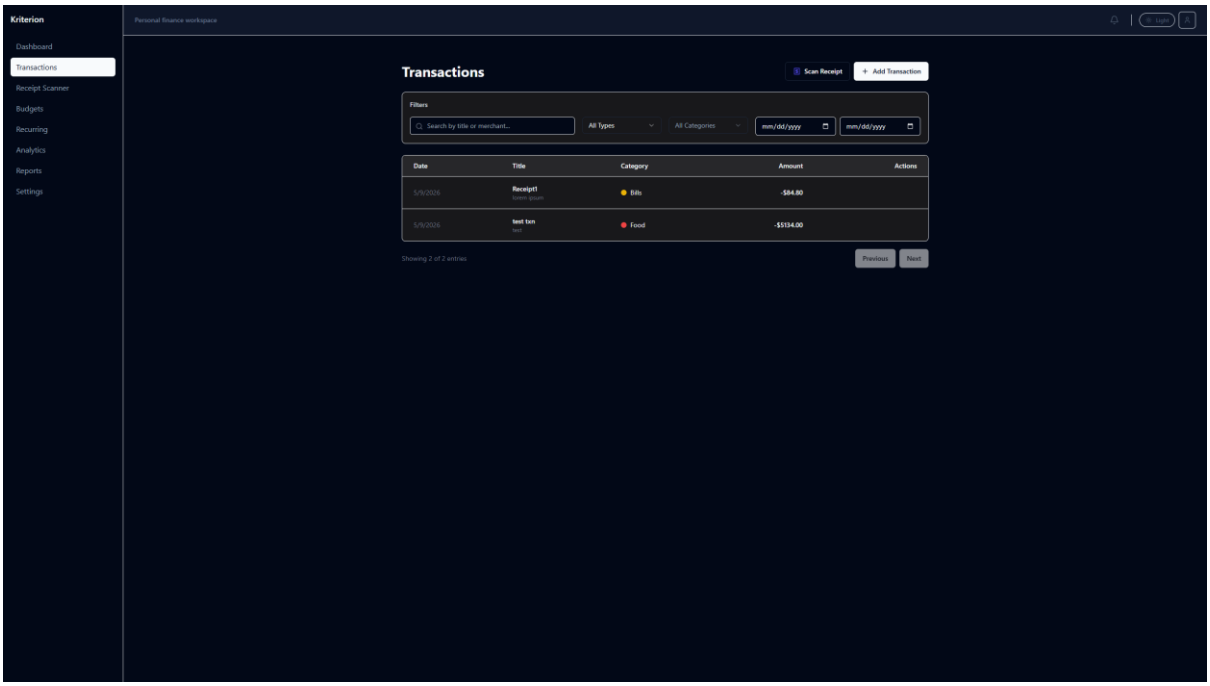


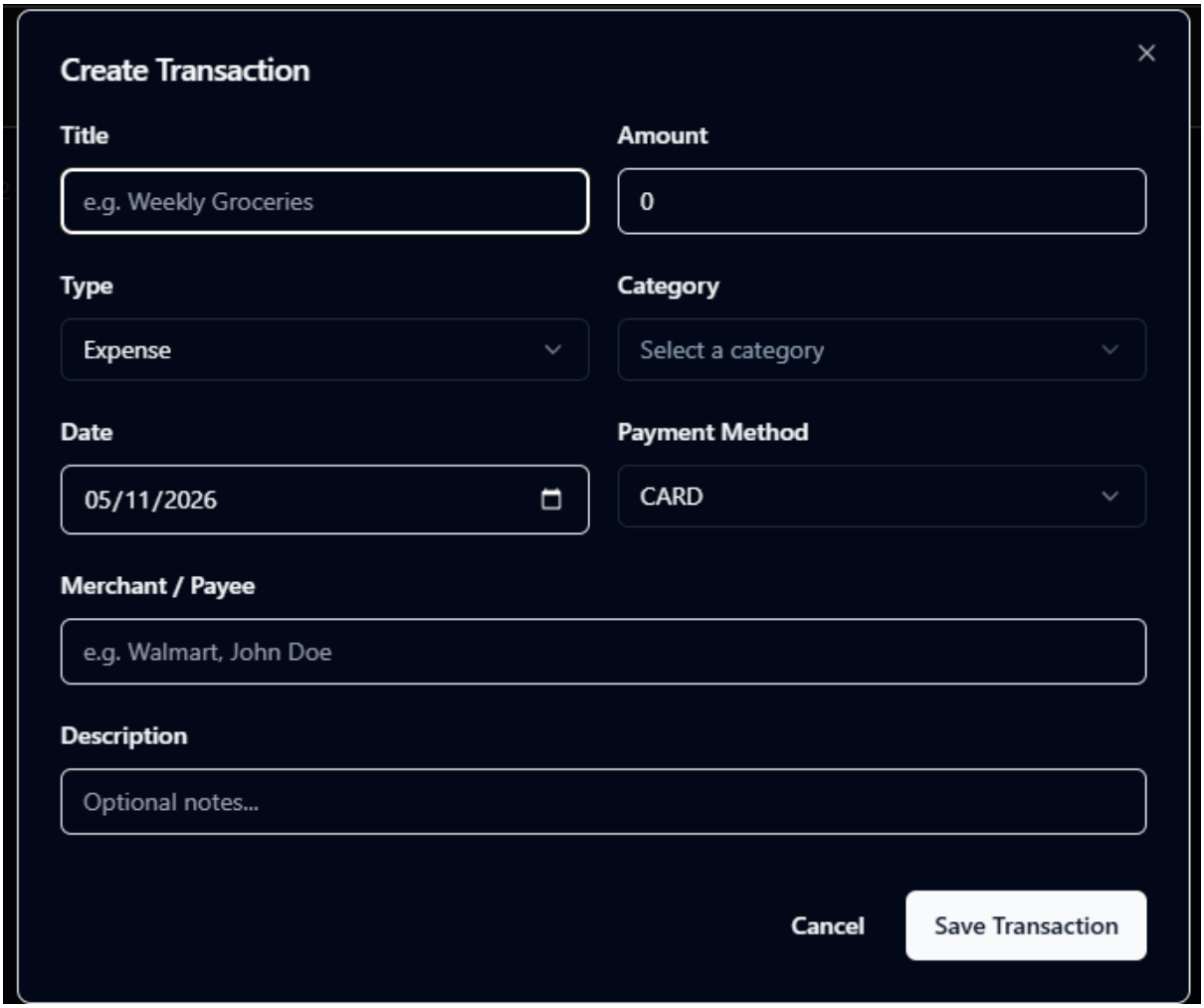
Managing Transactions:

1. Add New Transaction: Click the “Add Transaction” button, fill in details (amount,

category, date, description), and save. You can also upload a receipt for OCR processing.

2. View/Edit Transactions: Navigate to the “Transactions” section to see a list of all your records. Use filters and search to find specific transactions. Click on a transaction to edit or delete it.





Setting Budgets:

1. Create Budget: Go to the "Budgets" section, select a category (or overall expenses), set a monthly limit, and save.
2. Monitor Progress: The dashboard and budget section will show your spending against your set limits, with alerts if you are nearing or exceeding them.

Analyzing Finances:

1. Dashboard Charts: Use the interactive charts on the dashboard to visualize your spending patterns by category and over time.
2. Insights: Review the generated insights for personalized financial advice.

This concludes the Kriterion Project Report. The attached diagrams provide visual representations of the system's architecture, data model, and key processes.